

```
marks=[45,50,55,60,90]
mean=sum(marks)/len(marks)
print("Mean:",mean)
```

Mean: 60.0

Median

```
#it is only find by after sorting the data
#1.Arrange data in asc order
#2.find the mid val
```

```
import statistics
marks=[30,40,50,60,70]
median=statistics.median(marks)
print("Median:",median)
```

Median: 50

```
#When to use median?
#when data contains outliers
```

```
#example:
#salary=[20000,22000,25000,28000,500000]
```

#it is the val that appears most frequently

#Example

#data=[10,20,20,30,40]

#here:

#20 appears 2 times

#so mode = 20

```
import statistics
```

```
data=[10,20,20,30,40]
```

```
mode=statistics.mode(data)
```

```
print("Mode:",mode)
```

Mode: 20

#when to use

#finding the most common item

#ex:

#most common shirt size

```
import pandas as pd
```

```
df=pd.DataFrame({
```

```
    'Marks':[45,50,55,60,90]
```

```
})
```

```
print("Mean:")
```

```
print(df['Marks'].mean())
```

```
print("\nMedian:")
```

```
marks=[45,50,55,60,90]
mean=sum(marks)/len(marks)
print("Mean:",mean)
```

Mean: 60.0

Median

```
#it is only find by after sorting the data
#1.Arrange data in asc order
#2.find the mid val
```

```
import statistics
marks=[30,40,50,60,70]
median=statistics.median(marks)
print("Median:",median)
```

Median: 50

```
#When to use median?
#when data contains outliers
```

```
#example:
#salary=[20000,22000,25000,28000,500000]
```

```
import pandas as pd
df=pd.DataFrame({
    'Marks':[45,50,55,60,90]
})
print("Mean:")
print(df['Marks'].mean())
print("\nMedian:")
print(df['Marks'].median())
print("Mode:")
print(df['Marks'].mode())
```

Mean:
60.0

Median:
55.0

Mode:
0 45
1 50
2 55
3 60
4 90

Name: Marks, dtype: int64

Variance & Standard Deviation-Measure of speed

#Measures of spread tell us how far data val are spread from the centre(mean)

Two datasets can have the same mean but very diff spreads.

ex:a=[48,49,50,51,52]

b=[20,35,50,65,80]

Both have mean=50

but dataset b is much more spread out

```
import numpy as np
data=[10,12,14,16,18]
print(np.mean(data))
variance=np.var(data)
print("Variance:",variance)
```

14.0
Variance: 8.0

#Standard deviation

SD **is** the square root of variance

#Interpretation

Small SD -> Data points are close to mean

Large SD -> Data points are far **from** mean

#SD code

```
import numpy as np
data=[10,12,14,16,18]
print(np.mean(data))
variance=np.var(data)
std=np.std(data)
print("Variance:",variance)
print("Std:",std)
```

14.0
Variance: 8.0
Std: 2.8284271247461903

#Interpretil Range(IQR)

IQR meaasures the spread of the mid **50%** of data

It **is** less affected by outliers than variance **and** sd
formula:

$IQR = Q3 - Q1$

where

$Q1 = \text{First Quartile}(25\%)$

$Q3 = \text{Third Quartile}(75\%)$

$\text{data} = [5, 10, 15, 20, 25, 30, 35, 40]$

Quartiles

$Q1 = 13.75$

$Q3 = 31.25$

IQR

$IQR = 31.25 - 13.75 = 17.5$

```
] : ##IQR Code
import numpy as np
data=[5,10,15,20,25,30,35,40]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)
```

Q1: 13.75

Q3: 31.25

IQR: 17.5

```
] : ##IQR for Outlier Detection
```

Outliers are often detected using:

Lower limit= $Q1 - 1.5 \times IQR$

Upper limit= $Q3 - 1.5 \times IQR$

Any value outside these limits is considered an outlier.

#Code

```
import numpy as np
data=[10,12,14,15,16,18,100]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
lower=Q1-1.5*IQR
upper=Q3+1.5*IQR
print(lower)
print(upper)
outliers=[x for x in data if x < lower or x > upper]
print("Outliers:",outliers)

7.0
23.0
Outliers: [100]
```

#Probability basics

Probability **is** the measure of how likely an event **is** to occur

0=Impossible event

1=Certain event

Ex:

Probability of getting head **in** a coin toss=**0.5**

Probability of rolling a **6** on aa die=**1/6**

1.Sample space(**S**)

The **set** of **all** possible outcomes of an experiment

2.Event(**A**)

An event **is** a subset of the sample space.

3.Probability of an event **P(A)**

ex:

3. Probability of an event $P(A)$

ex:

Fine probability of getting an even no from a die

```
sample_space=[1,2,3,4,5,6]
```

```
event_A=[2,4,6]
```

```
P_A=len(event_A)/len(sample_space)
```

```
print("P(A)=",P_A)
```

4. Conditional probability $P(A|B)$ #or for intersection

Probability of A occurring given that B has already occurred

5. Bayes' Theorem

Used to calculate the probability of an event based on prior knowledge.

Ex:Disease Testing

Suppose

```
P_Disease=0.01
```

```
P_Positive_give_Disease=0.95
```

```
P_Positive=0.05
```

```
sample_space={1,2,3,4,5,6}
```

```
event_A={2,4,6} #even numbers #P(A) is not in python so we put _
```

```
print("Sample Space(S):",sample_space)
```

```
print("Event A:",event_A)
```

#2. Probability

```
P_A=len(event_A)/len(sample space)
```



```

print("\nProbability of Event A")
print("P(A) =", P_A)

#Conditional probability
#P(A|B)

event_B={4,5,6}    #Numbers greater than 3
intersection=event_A.intersection(event_B)
P_A_given_B=len(intersection) / len(event_B)

print("\nEvent B:", event_B)
print("A n B:", intersection)
print("P(A|B)=", P_A_given_B)

```

Sample Space(S): {1, 2, 3, 4, 5, 6}
 Event A: {2, 4, 6}

Probability of Event A
 $P(A) = 0.5$

Event B: {4, 5, 6}
 $A \cap B: \{4, 6\}$
 $P(A|B) = 0.6666666666666666$

#\$.Bayes Theorem code

```

#Ex:
#Disease prevalence=1%
#test sensitivity=99%
#Positive test probability=5%

```

#Pearson correlation(gradual inc)

Measures linear relationship btw 2 numerical variables(**for** proper relationships)

When to use pearson?

Numerical data

Linear relationship

Less outliers

Real life ex:

1.Height&weight

2.study hrs **and** marks

3.advertising spend **and** sales

```
import pandas as pd
df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,70,90]
})
corr=df['Hours'].corr(df['Marks'])
print("Pearson Correlation:",corr)
```

#close to +1

#strong positive relation

```

P_Disease=0.01
P_Positive_given_Disease=0.99
P_Positive=0.05

P_Disease_given_Positive=(
    P_Positive_given_Disease*P_Disease
)/P_Positive

print("\nBayes' Theorem Example")
print("P(Disease)=",P_Disease)
print("P(Positive|Disease)=",P_Positive_given_Disease)
print("P(Positive)=",P_Positive)

print(
    "P(Disease|Positive)=",
    round(P_Disease_given_Positive,4)
)
print(
    "Percentage=",
    round(P_Disease_given_Positive*100,2),
    "%")

```

```

Bayes' Theorem Example
P(Disease)= 0.01
P(Positive|Disease)= 0.99
P(Positive)= 0.05
P(Disease|Positive)= 0.198
Percentage= 19.8 %

```

Correlation

what **is** correlation?

Correlation measures the relationship btw **2** variables

Ex:

study hrs(increase) - marks(increase)

Used when:

Data **is not** normally distributed

Outliers exist

Relationship **not** perfectly linear

```
import pandas as pd
df=pd.DataFrame({
    'Rank_Study':[1,2,3,4,5],
    'Rank_Marks':[2,1,3,5,4]
})
corr=df['Rank_Study'].corr(
    df['Rank_Marks'],
    method='spearman'
)
print("Spearman Correlation:",corr)
```

Spearman Correlation: 0.7999999999999999

#Correlation matrix

```
import pandas as pd
df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,70,90],
    'Sleep':[8,7,6,5,4]
})
print(df.corr())
```

	Hours	Marks	Sleep
Hours	1.00000	0.99485	-1.00000
Marks	0.99485	1.00000	-0.99485
Sleep	-1.00000	-0.99485	1.00000

#Interpretation

Hours vs marks=0.99

Strong negative correlation

```
#Heatmap  
import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
df=pd.DataFrame({  
    'Hours': [1,2,3,4,5],  
    'Marks': [20,40,50,70,90],  
    'Sleep': [8,7,6,5,4]  
})  
sns.heatmap(df.corr(),  
    annot=True,  
    cmap='YlGnBu')  
plt.show()
```

#Common cmap values

#coolwarm

#viridis

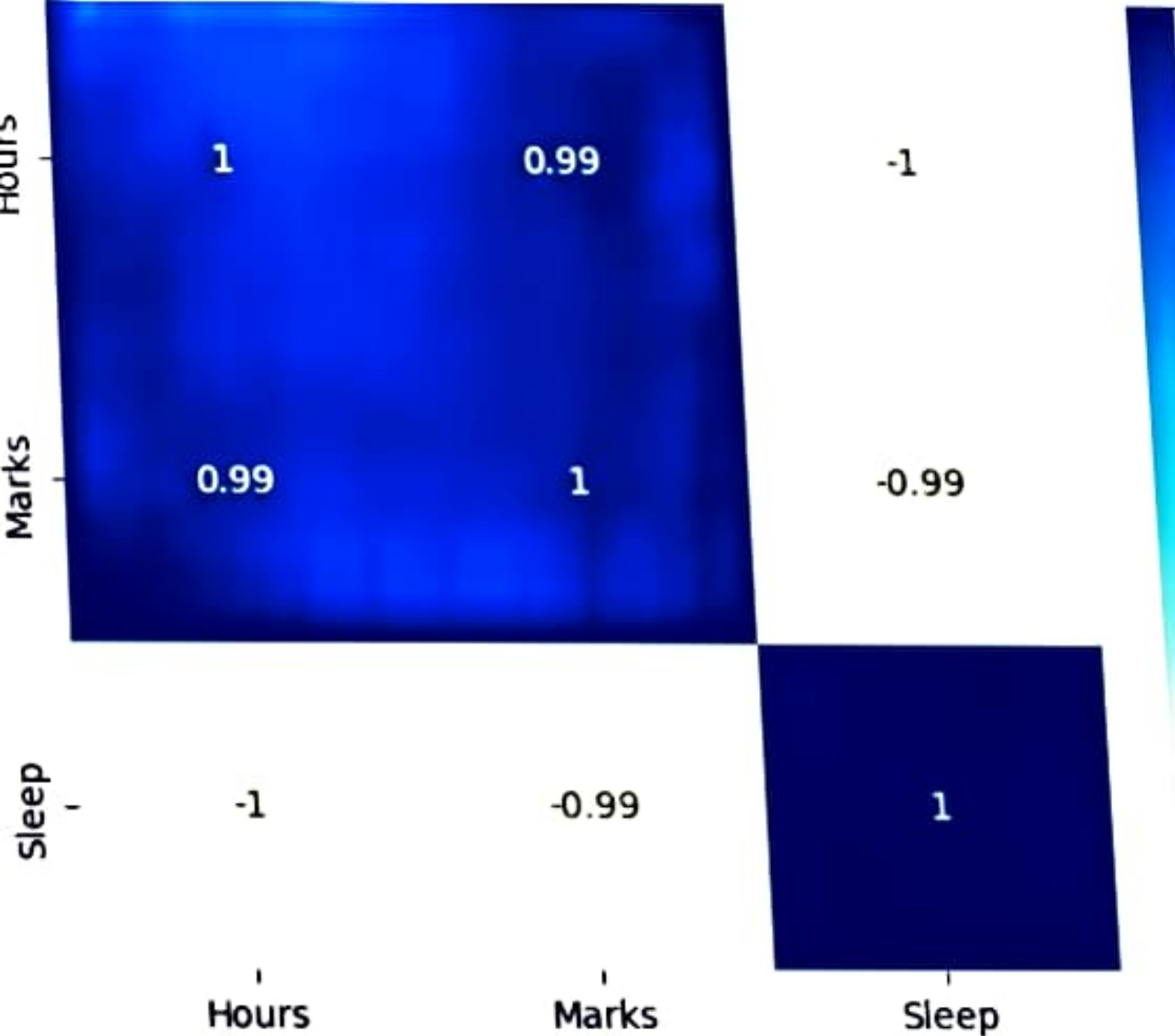
#YlGnBu

#Blues

#Red

#greens

#magma



```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

#Generate normally distributed data
data=np.random.normal(
    loc=70,    #Mean
    scale=10,  #SD
    size=1000 #No of values
)
```

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

#Generate normally distributed data
data=np.random.normal(
    loc=70,    #Mean
    scale=10,  #SD
    size=1000 #No of values
)

#Calculate mean and standard deviation
mean=np.mean(data)
std=np.std(data)

print("Mean:",round(mean,2))
print("Standard deviation:",round(std,2))

#Plot histogram
plt.figure(figsize=(8,5))
sns.histplot(data,bins=30,kde=True)

plt.title("Normal Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")

plt.show()
```

Mean: 70.31

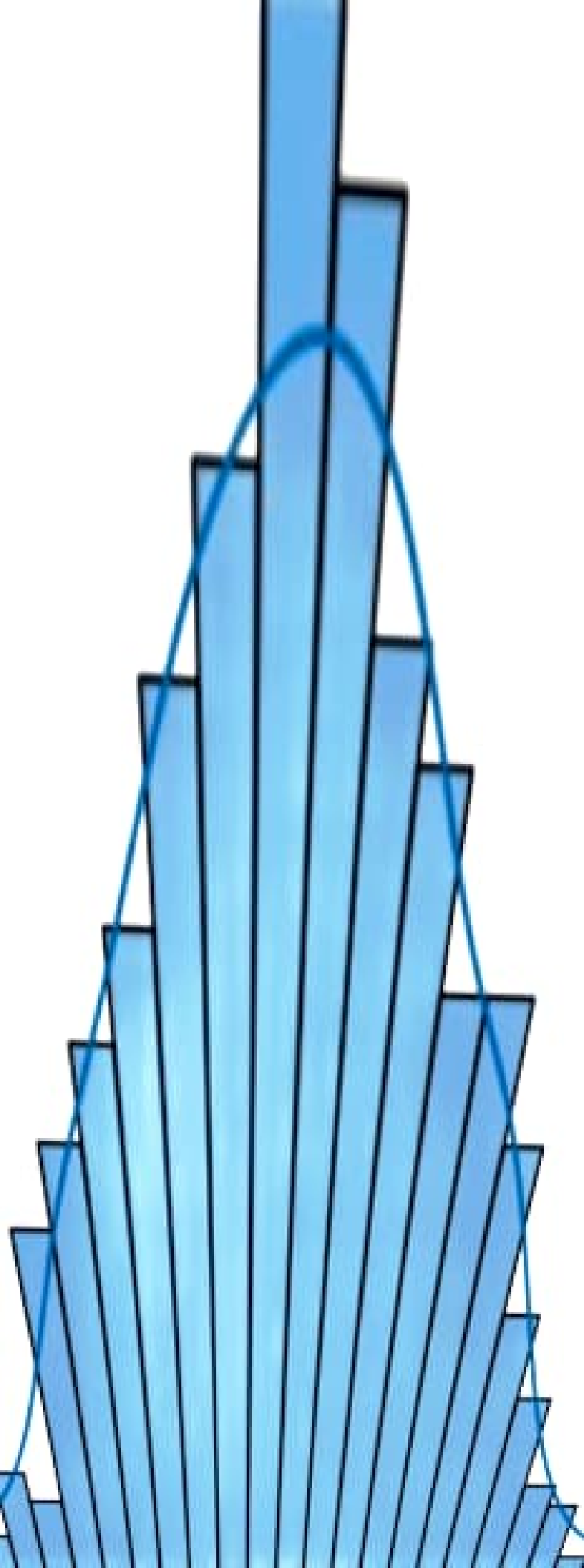
Standard deviation: 9.8


```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

#Load dataset
df=sns.load_dataset("diamonds")

#Histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)

plt.title("Distribution of Diamond Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```



```
[6]: import numpy as np
      scores=[60,65,70,75,80]
      mean=np.mean(scores)
      std=np.std(scores)

      score=80

      z_score=(score-mean)/std

      print("Mean:",mean)
      print("Standard Deviation:",std)
      print("Z-score:",z_score)

      Mean: 70.0
      Standard Deviation: 7.0710678118654755
      Z-score: 1.414213562373095
```

```
[ ]:
```